EX: 10 Air Quality Prediction and Pollution Monitoring System

Problem Statement:

Air pollution is one of the most serious environmental and public-health problems of modern cities. The quality of the air depends on the concentration of several pollutants such as $PM_{2.5}$, PM_{10}, NO_2, SO_2, CO_2, and O_3, together with weather conditions such as temperature and humidity. Checking these readings manually and deciding how polluted the air is can be slow and may not capture the combined effect of all these factors.

There is, therefore, a need for a machine learning based system that can analyze the pollutant and weather readings and predict the pollution level (Low, Moderate or High). This project develops an air quality prediction and pollution monitoring system using Logistic Regression and Random Forest Classification, with evaluation based on Accuracy, Precision, Recall, F1-Score and the Confusion Matrix. The trained Random Forest model is also deployed in a Flask web application with a SQLite database, where a user can enter the readings, instantly get the predicted pollution level, view a monitoring dashboard, browse the dataset and review the prediction history.

Objectives:

1. To load and understand the air quality dataset.
2. To preprocess the dataset by handling missing values and duplicate records.
3. To select the input features (PM2.5, PM10, NO2, SO2, CO, O3, temperature, humidity) and the target variable (pollution level).
4. To perform exploratory data analysis and visualize important relationships.
5. To train Logistic Regression and Random Forest classification models.
6. To evaluate the models using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix.
7. To predict the pollution level for new readings, save the trained model, and monitor predictions through the web application.

Software / Hardware Required:

- Operating System: Windows / Linux / macOS
- IDE: Visual Studio Code / Jupyter Notebook
- Language: Python 3.x
- Machine Learning: scikit-learn
- Data Processing: pandas, NumPy
- Data Visualization: Matplotlib
- Web Framework: Flask, with HTML, CSS and JavaScript for the front end
- Database: SQLite (database/air_quality.db)
- Dataset: air_quality_50.csv (50 records, 8 input features and 1 target column)
- Model Saving: joblib

Project Directory Structure:

The project is organized as a Python machine learning project containing the dataset, source programs, trained model, database and the web application.

```
air_quality_prediction/
├── data/air_quality_50.csv
├── train_model.py
├── models/air_quality_model.joblib
├── reports/model_metrics.json
├── app.py
├── database.py
├── database/air_quality.db
├── requirements.txt
├── templates/
└── static/
```

Procedure:

1. Load the air quality dataset using pandas.
2. Display the dataset shape, column names, information, and descriptive statistics.
3. Check missing values and remove missing and duplicate records.
4. Visualize the pollution level distribution, PM2.5 versus PM10, and PM2.5 for each pollution level.
5. Separate the input features from the target variable pollution level.
6. Split the data into 80% training and 20% testing sets (stratified).
7. Train the Logistic Regression model and calculate Accuracy, Precision, Recall and F1-Score.
8. Train the Random Forest Classification model using 200 estimators and calculate the evaluation metrics.
9. Compare the models, analyze the confusion matrix and feature importance, and predict the pollution level for new readings.
10. Save the trained Random Forest model as air_quality_model.joblib and use it in the Flask web application.

Implementation Code:

1. Import the Required Libraries

```
import os
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.base import clone
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix
```

2. Load the Dataset

```
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)

df = pd.read_csv("data/air_quality_50.csv")
print("Dataset loaded successfully.")
print(df.head())
```

3. Understand the Dataset

```
print("Dataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns)
print("\nDataset Information:")
df.info()
print("\nStatistical Summary:")
print(df.describe())
```

4. Check Missing Values and Clean the Data

```
print("Missing Values:")
print(df.isnull().sum())
df = df.dropna()
df = df.drop_duplicates()
print("Shape after cleaning:", df.shape)
```

5. Basic Data Visualization

```
class_order = ["Low", "Moderate", "High"]
class_colors = {"Low": "green", "Moderate": "orange", "High": "red"}

plt.figure(figsize=(8, 5))
counts = df["Pollution_Level"].value_counts().reindex(class_order)
counts.plot(kind="bar", color=[class_colors[c] for c in class_order])
plt.title("Pollution Level Distribution")
plt.xlabel("Pollution Level")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()

plt.figure(figsize=(8, 5))
for level in class_order:
    part = df[df["Pollution_Level"] == level]
    plt.scatter(part["PM2_5"], part["PM10"], alpha=0.7,
                color=class_colors[level], label=level)
plt.title("PM2.5 vs PM10 by Pollution Level")
plt.xlabel("PM2.5 (µg/m3)")
plt.ylabel("PM10 (µg/m3)")
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
groups = [df[df["Pollution_Level"] == c]["PM2_5"] for c in class_order]
plt.boxplot(groups)
plt.xticks([1, 2, 3], class_order)
plt.title("PM2.5 Concentration by Pollution Level")
plt.xlabel("Pollution Level")
plt.ylabel("PM2.5 (µg/m3)")
plt.show()
```

6. Select Input Features and Target

```
features = ["PM2_5", "PM10", "NO2", "SO2", "CO", "O3", "Temperature", "Humidity"]
class_map = {"Low": 0, "Moderate": 1, "High": 2}

X = df[features]
y = df["Pollution_Level"].map(class_map)
print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

7. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

8. Logistic Regression Model

```
log_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000, class_weight="balanced",
                                      random_state=42))
])
log_model.fit(X_train, y_train)
log_predictions = log_model.predict(X_test)
print("Logistic Regression trained successfully.")
```

9. Evaluate Logistic Regression

```
log_acc = accuracy_score(y_test, log_predictions)
log_prec = precision_score(y_test, log_predictions, average="macro", zero_division=0)
log_rec = recall_score(y_test, log_predictions, average="macro", zero_division=0)
log_f1 = f1_score(y_test, log_predictions, average="macro", zero_division=0)
print("Logistic Regression Results")
print("Accuracy :", round(log_acc, 4))
print("Precision:", round(log_prec, 4))
print("Recall   :", round(log_rec, 4))
print("F1-Score :", round(log_f1, 4))
```

10. Random Forest Model

```
rf_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", RandomForestClassifier(n_estimators=200, class_weight="balanced",
                                          random_state=42))
])
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)
print("Random Forest trained successfully.")
```

11. Evaluate Random Forest

```
rf_acc = accuracy_score(y_test, rf_predictions)
rf_prec = precision_score(y_test, rf_predictions, average="macro", zero_division=0)
rf_rec = recall_score(y_test, rf_predictions, average="macro", zero_division=0)
rf_f1 = f1_score(y_test, rf_predictions, average="macro", zero_division=0)
print("Random Forest Results")
print("Accuracy :", round(rf_acc, 4))
print("Precision:", round(rf_prec, 4))
print("Recall   :", round(rf_rec, 4))
print("F1-Score :", round(rf_f1, 4))
```

12. Compare Models

```
results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [log_acc, rf_acc],
    "Precision": [log_prec, rf_prec],
    "Recall": [log_rec, rf_rec],
    "F1-Score": [log_f1, rf_f1]
})
print(results)

cm = confusion_matrix(y_test, rf_predictions, labels=[0, 1, 2])
print("\nConfusion Matrix (Low, Moderate, High):")
print(cm)

plt.figure(figsize=(6, 5))
plt.imshow(cm, cmap="Blues")
plt.colorbar()
plt.xticks([0, 1, 2], class_order)
plt.yticks([0, 1, 2], class_order)
for i in range(3):
    for j in range(3):
        txt_color = "white" if cm[i, j] > cm.max() / 2 else "black"
        plt.text(j, i, cm[i, j], ha="center", va="center", color=txt_color)
plt.title("Confusion Matrix (Random Forest)")
plt.xlabel("Predicted Level")
plt.ylabel("Actual Level")
plt.show()
```

13. Feature Importance

```
rf_classifier = rf_model.named_steps["classifier"]
importances = pd.Series(rf_classifier.feature_importances_,
                        index=features).sort_values()
plt.figure(figsize=(8, 5))
importances.plot(kind="barh")
plt.title("Feature Importance (Random Forest)")
plt.show()
```

14. Predict Pollution Level for New Data

```
new_data = pd.DataFrame({
    "PM2_5": [120.0],
    "PM10": [210.0],
    "NO2": [60.0],
    "SO2": [30.0],
    "CO": [1.5],
    "O3": [90.0],
    "Temperature": [28.0],
    "Humidity": [55.0]
})
pred_index = int(rf_model.predict(new_data)[0])
pred_proba = rf_model.predict_proba(new_data)[0]
print("Predicted Pollution Level:", class_order[pred_index])
for name, p in zip(class_order, pred_proba):
    print(name, "probability:", round(float(p), 4))
```

15. Save the Trained Model

```
final_model = clone(rf_model)
final_model.fit(X, y)


os.makedirs("models", exist_ok=True)
joblib.dump({
    "pipeline": final_model,
    "feature_columns": features,
    "class_names": class_order
}, "models/air_quality_model.joblib")
print("Model saved as models/air_quality_model.joblib")
```

Output (Step-by-Step):

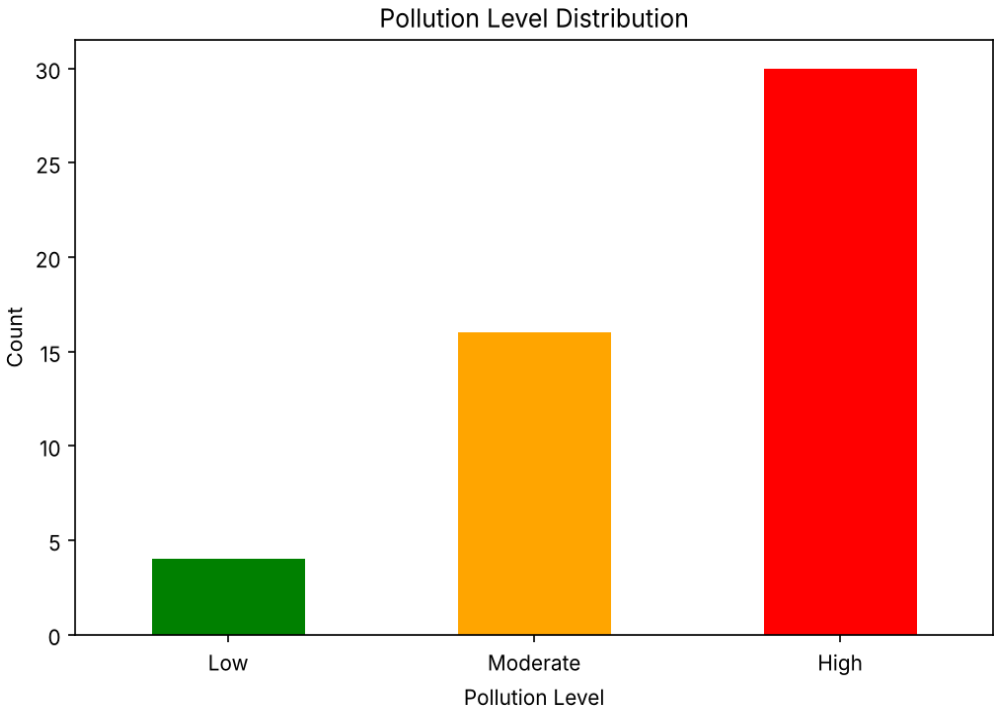


Fig 1: Distribution of records by pollution level.

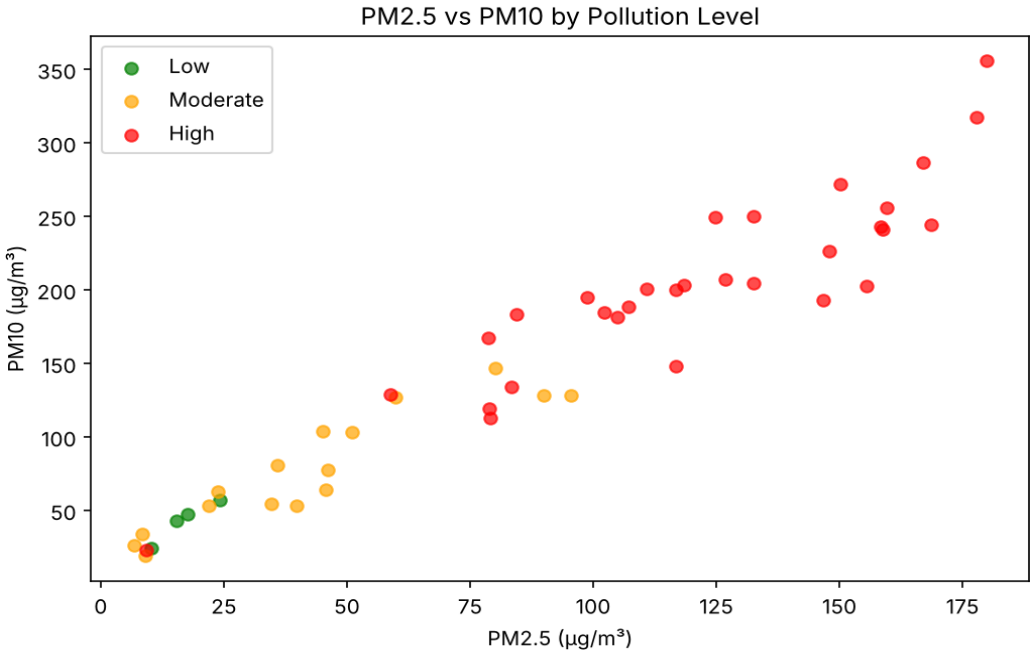


Fig 2: PM2.5 versus PM10 for each pollution level.

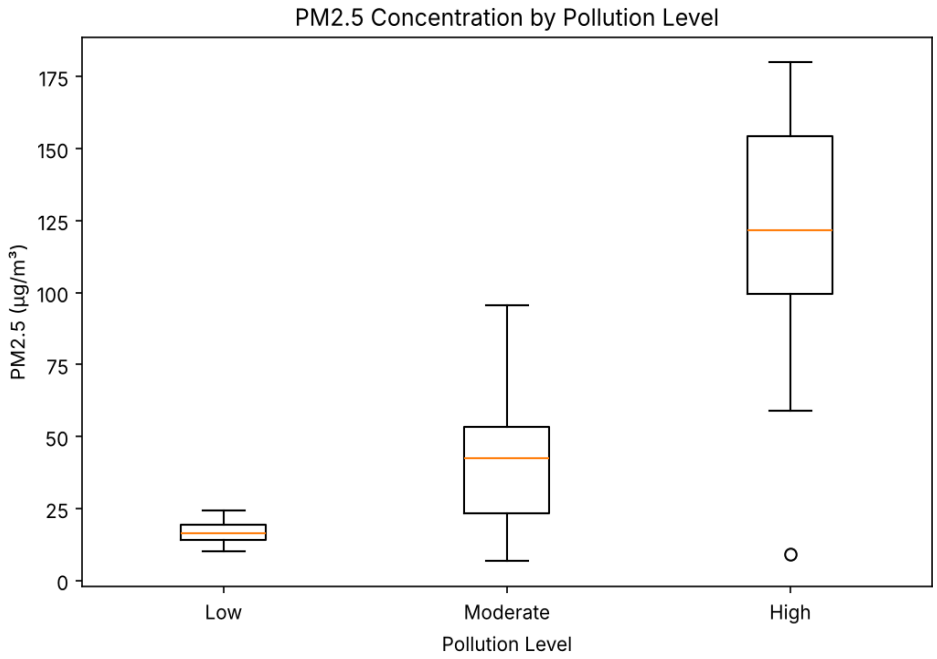


Fig 3: PM2.5 concentration for each pollution level.

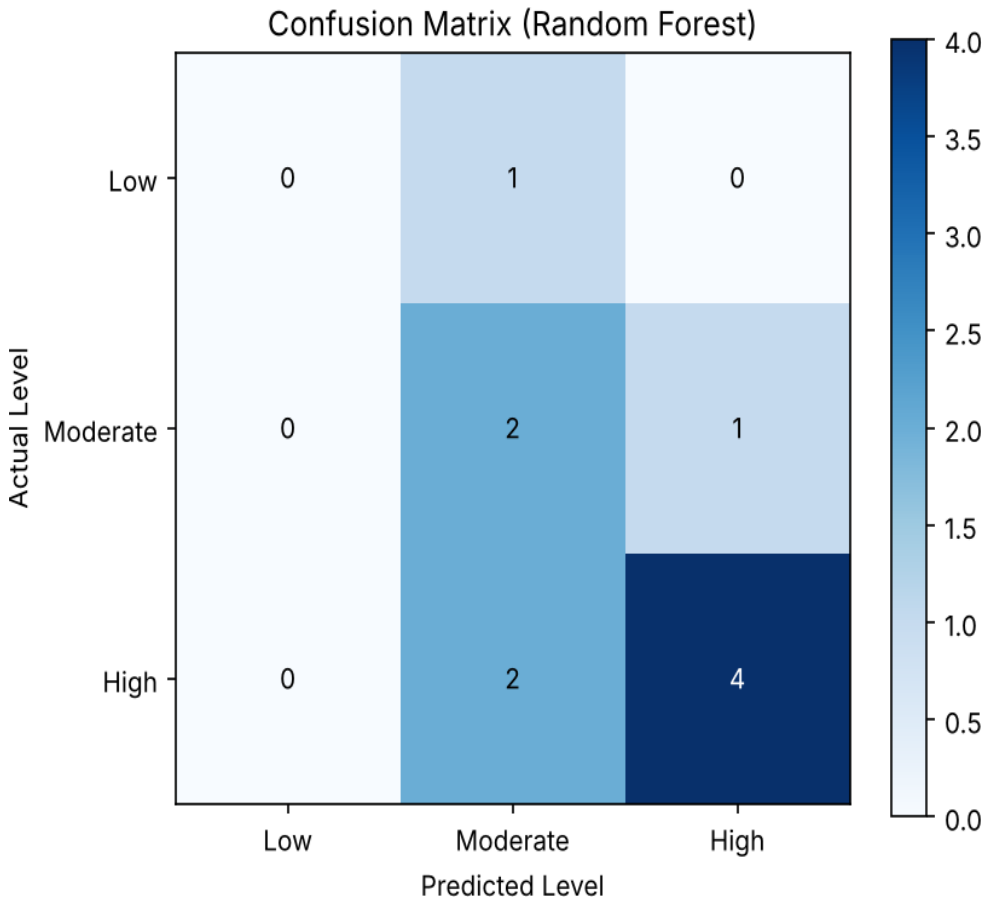


Fig 4: Confusion matrix of the Random Forest model.

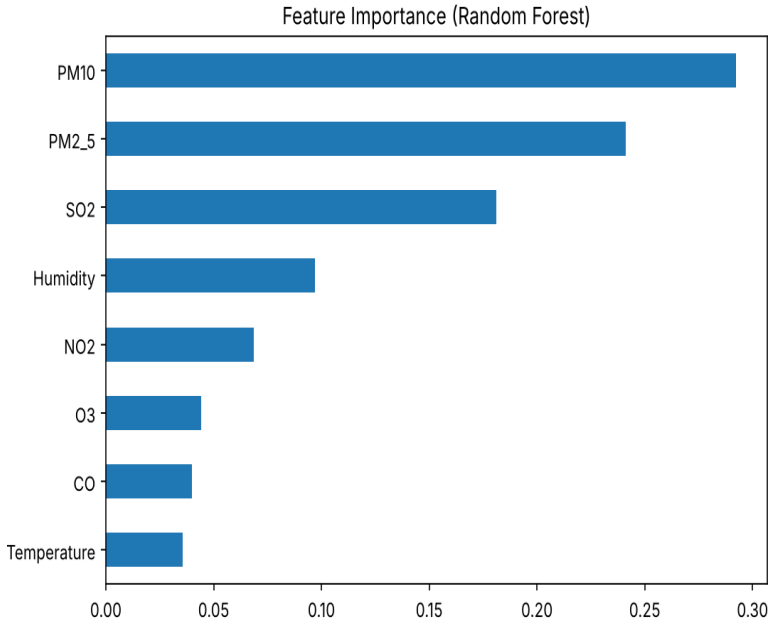


Fig 5: Feature importance of the Random Forest model.

Dataset Output:

- Dataset Shape: (50, 9)
- Columns: PM2_5, PM10, NO2, SO2, CO, O3, Temperature, Humidity, Pollution_Level
- Shape after cleaning: (50, 9)
- Class distribution: Low = 4, Moderate = 16, High = 30

Training and Testing Output:

- Training samples: 40
- Testing samples: 10

Model Performance:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7000	0.4444	0.5000	0.4683
Random Forest	0.6000	0.4000	0.4444	0.4091

Confusion Matrix (Random Forest) – rows: actual, columns: predicted (Low, Moderate, High):

	Low	Moderate	High
Low	0	1	0
Moderate	0	2	1
High	0	2	4

New Data Prediction:

PM2.5: 120 | PM10: 210 | NO2: 60 | SO2: 30 | CO: 1.5 | O3: 90 | Temperature: 28 °C | Humidity: 55 %

Predicted Pollution Level: High (probability: Low 0.000, Moderate 0.035, High 0.965)

Model Saving Output:

The trained Random Forest model was successfully saved for future predictions.

Output: Model saved successfully.

Result:

The Air Quality Prediction and Pollution Monitoring System was successfully implemented using Python, Machine Learning, Flask, and SQLite. Logistic Regression and Random Forest models were trained and evaluated. The system predicts pollution levels as Low, Moderate, or High based on pollutant and weather readings and displays the results through a web application.